

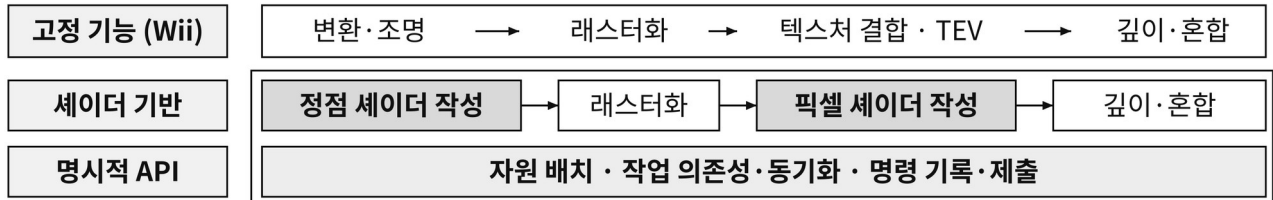
# 데이터 지향 설계와 명시적 GPU 제어 프레임워크 dy\_framework

|      |                       |     |             |
|------|-----------------------|-----|-------------|
| 팀 명  | DyD                   | 학 과 | 컴퓨터소프트웨어공학과 |
| 팀 원  | 한재승(팀장), 윤훈, 정현진, 정준혁 |     |             |
| 지도교수 | 정석용                   |     |             |

## 1. 작품의 개요

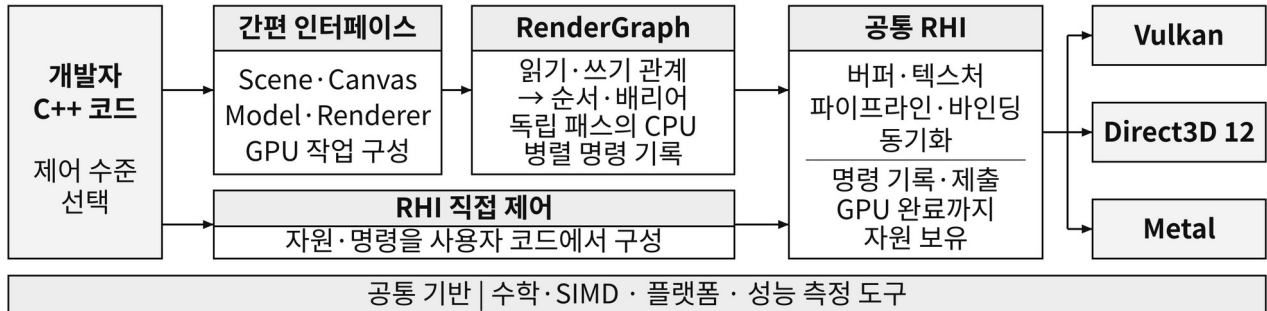
GPU는 그래픽스를 넘어 인공지능의 학습·추론과 과학 시뮬레이션을 수행하는 핵심 연산 기반으로 확장되고 있다. 이에 따라 하드웨어의 병렬 처리 능력과 메모리 구조를 소프트웨어 설계에 반영해야 한다. 데이터의 배치·접근·변환을 우선하는 데이터 지향 설계(DOD)는 이 성능을 활용하는 출발점이다.

현대 그래픽스 API는 셰이더의 연산뿐 아니라 GPU 자원의 사용 시점, 작업 간 의존성과 명령 제출까지 개발자의 설계 영역으로 확장했다.



기존의 공통 API와 암묵적 자원 관리에 맞춘 통합 코드만으로는 이 제어 모델을 대체하기 어렵다. Vulkan·Direct3D 12·Metal의 자원·명령 체계에 대응하면서 코드가 다시 API·플랫폼별로 갈라지고, 공통 설계를 플랫폼마다 반복해서 구현해야 한다. dy\_framework는 이를 공통 RHI에서 다루며, 데이터의 준비와 처리부터 GPU 자원과 실행 명령까지 연결하는 멀티플랫폼 C++ 프레임워크이다.

## 2. 작품의 구성 및 동작 설명



간편 인터페이스로 장면과 모델을 구성하거나 RHI를 직접 제어할 수 있다. RenderGraph는 패스의 의존성과 기록 순서를 구성하고, RHI는 명령 제출과 GPU 완료까지의 자원 보유를 맡는다.

## 3. 프로젝트 개발환경 및 개발자

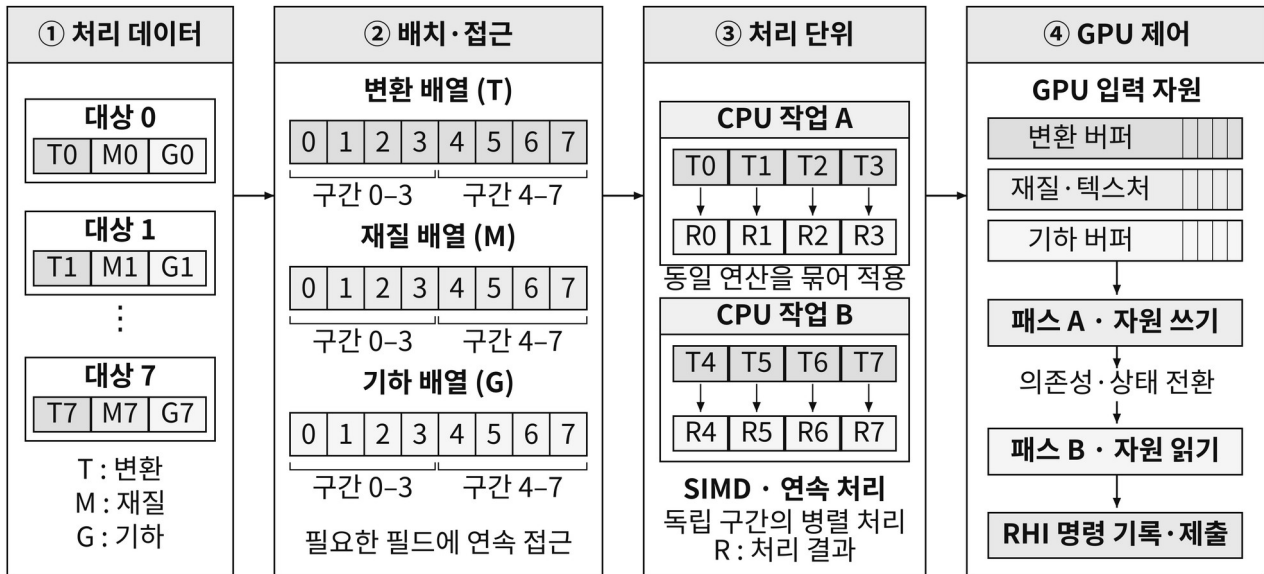
개발도구 및 사용 언어 :  
C++17, CMake  
MSVC / GCC / Clang  
HLSL, GLSL, Metal  
Windows, Linux, macOS

|                    |                     |                         |                     |
|--------------------|---------------------|-------------------------|---------------------|
|                    |                     |                         |                     |
| 한재승<br>Core<br>RHI | 윤훈<br>Vulkan<br>RHI | 정현진<br>DirectX12<br>RHI | 정준혁<br>Metal<br>RHI |

## 4. 핵심기술 설명

### 4-1. 데이터 지향 설계(DOD)에서 명시적 GPU 제어로

처리할 데이터의 배치·접근·변환을 먼저 설계하고, 그 흐름을 GPU 자원과 실행 명령으로 연결한다.



### 4-2. 명시적 GPU 제어부터 고급 렌더링까지

자원의 배치와 접근, 작업의 의존성과 제출을 함께 다루는 관점은 GPU 기반 처리와 렌더링 기법으로 이어진다.

#### GPU 실행 · 데이터 제어

##### RenderGraph / Resource Barriers

자원의 읽기·쓰기 관계를 바탕으로 작업 의존성과 상태 전환을 구성한다.

##### Multithread Command Recording

독립 작업의 RHI 명령과 네이티브 GPU 명령을 여러 CPU 스레드에서 병렬 기록하고, 의존 순서에 맞춰 제출한다.

##### Instancing & Batching

공유 기하와 객체별 속성 배열을 활용하고, 호환되는 작업을 묶어 CPU의 반복 제출을 줄인다.

##### Bindless Materials

리소스 집합의 바인딩을 유지하면서 셰이더의 인덱스로 재질·텍스처를 선택한다.

##### Async Compute

그래픽스와 컴퓨트 작업을 별도 큐에 배치하고, 의존성을 조정해 실행 중첩을 구성한다.

#### GPU 기반 처리 · 렌더링

##### Indirect Drawing / GPU Culling

GPU가 가시성을 판정하고 그리기 인자를 구성하여, 처리 대상을 GPU 실행 흐름 안에서 결정한다.

##### Mesh Shader

작업 그룹 단위로 정점과 프리미티브를 생성·처리하여 기하 처리 흐름을 구성한다.

##### Meshlet Culling / LOD

메시를 작은 묶음으로 나누고 가시성과 상세도를 선택해 처리할 기하 데이터의 범위를 줄인다.

##### Clustered Lighting

공간을 클러스터로 나누고 광원을 분류해, 각 영역에 필요한 광원 데이터를 참조한다.

##### Temporal Anti-Aliasing

이전 프레임 정보를 재투영·누적해 시간에 걸쳐 영상의 계단 현상을 줄인다.

##### Volumetric Fog

시선 경로의 산란과 감쇠를 계산해 공간 속 빛과 안개를 표현한다.